TASKTRACKER

Planejamento Lógico e Estruturação do Projeto

Bootcamp II - Desafio Cumulativo - Fase 11

Nome completo	Alberani Gertrudes Filho
Curso	Ciência de Dados
Polo / Turma	Asa Norte
E-mail institucional	alberani.gertrudes@sempreceub.com

Documento-base para as próximas etapas do projeto (Git/GitHub, Python e Docker)

Objetivo deste documento

Definir, antes da programação, o problema que o TaskTracker resolve, os dados que o sistema receberá e exibirá, as regras de funcionamento, o algoritmo em português e a organização inicial do repositório que será usada na Fase 2.

1. Apresentação e Entendimento do Problema

1.1 Identificação

Aluno: Alberani Gertrudes Filho
Curso: Ciência de Dadoss
Polo/Turma: Asa Norte
E-mail institucional: alberani.gertrudes@sempreceub.com

1.2 O problema a ser resolvido

No dia a dia, uma pessoa pode acumular tarefas de estudo, trabalho e compromissos pessoais em lugares diferentes, como mensagens, anotações, agenda e memória. Isso dificulta enxergar o que ainda precisa ser feito, quais tarefas são mais importantes e quais têm prazo mais próximo. Sem um sistema simples de organização, tarefas podem ser esquecidas, executadas fora de prioridade ou concluídas sem um registro claro do andamento.

A principal dor que o projeto pretende resolver é a falta de uma visão única e objetiva das tarefas. O usuário precisa conseguir registrar rapidamente uma atividade, identificar sua prioridade e prazo, consultar o que está pendente e marcar o que já foi concluído.

1.3 Solução proposta: TaskTracker

O TaskTracker será um gerenciador de tarefas simples, inicialmente executado em Python na Fase 2. O sistema permitirá cadastrar tarefas, visualizar a lista organizada e concluir tarefas já cadastradas. Cada tarefa terá informações suficientes para facilitar a tomada de decisão do usuário sem tornar o programa complexo demais para a primeira versão.

Funcionalidades previstas na primeira versão

Função	O que resolve
Cadastrar tarefa	Registrar título, descrição, prioridade e data limite.
Visualizar tarefas	Exibir as tarefas com identificação, prioridade, prazo e status.
Organizar a leitura	Apresentar pendências antes das concluídas e destacar prioridades/prazos.
Concluir tarefa	Alterar o status de uma tarefa existente de Pendente para Concluída.
Informar o usuário	Exibir mensagens claras de sucesso, lista vazia ou dados inválidos.

2. Decomposição do Problema e Dados do Sistema

2.1 Entradas de dados

Entrada	Finalidade	Exemplo
Opção do menu	Número que representa a ação desejada pelo usuário.	1, 2, 3 ou 4
Título da tarefa	Nome curto da tarefa. Campo obrigatório.	Entregar atividade do Bootcamp

Entrada	Finalidade	Exemplo
Descrição	Detalhamento opcional da tarefa.	Revisar documento e enviar PDF
Prioridade	Grau de importância definido pelo usuário.	Alta, Média ou Baixa
Data limite	Prazo previsto para conclusão.	20/09/2026
ID da tarefa	Identificador informado quando o usuário deseja concluir uma tarefa.	3

2.2 Saídas de dados

- Mensagem de confirmação após o cadastro de uma tarefa.
- Lista de tarefas com ID, título, prioridade, data limite e status.
- Quantidade de tarefas pendentes e concluídas.
- Mensagem informando quando ainda não existem tarefas cadastradas.
- Mensagem de erro quando o usuário informa opção, prioridade, data ou ID inválido.
- Mensagem de confirmação quando uma tarefa é marcada como concluída.

2.3 Regras do sistema

Regra	Definição
R1 - Título obrigatório	O sistema não deve cadastrar uma tarefa sem título. Se o campo estiver vazio, deve solicitar novamente.
R2 - Prioridade controlada	A prioridade deve aceitar apenas Alta, Média ou Baixa. Qualquer outro valor deve ser rejeitado e solicitado novamente.
R3 - Data válida	A data limite deve estar em formato válido (DD/MM/AAAA) e não pode ser anterior à data do cadastro.
R4 - Status inicial	Toda nova tarefa deve ser criada automaticamente com o status Pendente.
R5 - Identificador único	Cada tarefa deve receber um ID numérico único e crescente para permitir sua localização.
R6 - Conclusão segura	Somente uma tarefa existente pode ser concluída. Se o ID não existir, o sistema deve avisar o usuário sem alterar a lista.
R7 - Organização da visualização	Na listagem, tarefas pendentes devem aparecer antes das concluídas; entre as pendentes, prioridade e prazo devem facilitar a leitura do que é mais urgente.

3. Receita Lógica do Sistema - Algoritmo em Português

A lógica abaixo descreve o funcionamento do TaskTracker sem usar sintaxe de programação. A ideia é que qualquer pessoa consiga acompanhar o fluxo e que esse texto sirva de base direta para a implementação em Python na Fase 2.

3.1 Fluxo geral

1. Iniciar o sistema com uma lista de tarefas vazia e o próximo ID igual a 1.
2. Exibir um menu com as opções: 1 - Cadastrar tarefa; 2 - Visualizar tarefas; 3 - Concluir tarefa; 4 - Sair.
3. Ler a opção escolhida pelo usuário.
4. Executar o fluxo correspondente à opção escolhida.
5. Ao terminar a ação, voltar ao menu principal.
6. Repetir o processo até o usuário escolher a opção Sair.
7. Encerrar o programa com uma mensagem de finalização.

3.2 Passo a passo do cadastro

8. Pedir ao usuário o título da tarefa.
9. Verificar se o título não está vazio. Se estiver vazio, avisar e pedir novamente.
10. Pedir uma descrição da tarefa; permitir que esse campo fique vazio.
11. Pedir a prioridade da tarefa.
12. Verificar se a prioridade é Alta, Média ou Baixa. Se não for, avisar e pedir novamente.
13. Pedir a data limite no formato DD/MM/AAAA.
14. Verificar se a data é válida e se não é anterior à data atual. Se for inválida, pedir novamente.
15. Criar a tarefa com o próximo ID disponível e status Pendente.
16. Adicionar a tarefa à lista.
17. Aumentar o próximo ID em 1.
18. Exibir uma mensagem confirmando o cadastro e informar o ID da nova tarefa.

3.3 Passo a passo da visualização

19. Acessar a lista de tarefas.
20. Verificar se a lista está vazia.
21. Se estiver vazia, informar que ainda não existem tarefas cadastradas e voltar ao menu.
22. Se houver tarefas, separar ou ordenar a exibição para mostrar primeiro as pendentes e depois as concluídas.
23. Entre as tarefas pendentes, priorizar a leitura por prioridade (Alta, Média, Baixa) e, quando houver empate, pelo prazo mais próximo.
24. Exibir, para cada tarefa, o ID, o título, a prioridade, a data limite e o status.
25. Exibir ao final o total de tarefas pendentes e concluídas.
26. Voltar ao menu principal.

3.4 Passo a passo para concluir uma tarefa

27. Verificar se existe pelo menos uma tarefa cadastrada. Se não existir, avisar o usuário e voltar ao menu.
28. Mostrar a lista de tarefas e pedir o ID da tarefa que será concluída.
29. Procurar a tarefa com o ID informado.
30. Se o ID não existir, exibir uma mensagem de erro e voltar ao menu sem alterar nenhuma tarefa.
31. Se a tarefa já estiver Concluída, informar que ela já foi finalizada.
32. Se a tarefa estiver Pendente, alterar seu status para Concluída.
33. Exibir uma mensagem de sucesso com o título da tarefa concluída.
34. Voltar ao menu principal.

3.5 Tratamento de escolhas inválidas

Se o usuário digitar uma opção que não existe no menu, o sistema deve informar que a escolha é inválida e exibir o menu novamente. O mesmo princípio será aplicado aos campos controlados, como prioridade, data e ID: nenhum dado inválido deve alterar a lista de tarefas.

4. Mapeamento Arquitetural para Git e GitHub

Na Fase 2, o planejamento desta etapa será transformado em arquivos versionados no GitHub. A estrutura inicial do repositório será simples para separar documentação e código-fonte.

4.1 Desenho da árvore do repositório

```
tasktracker-projeto/
├── README.md                 # Apresentação do projeto no GitHub
├── .gitignore                # Arquivos temporários que não devem ser versionados
```

```
├── docs/
│   └── planejamento_logico.pdf # Documento de planejamento produzido na Fase 1
└── src/
    └── main.py                 # Arquivo principal do programa em Python
```

4.2 Função de cada item

Arquivo/Pasta	Responsabilidade
README.md	Apresentará o objetivo do TaskTracker, como o projeto está organizado e, futuramente, como executar o programa.
.gitignore	Evitará o envio de arquivos temporários, caches e outros arquivos desnecessários ao repositório.
docs/	Guardará a documentação do projeto, começando pelo planejamento lógico desta Fase 1.
src/	Concentrará o código-fonte da aplicação.
src/main.py	Será o ponto principal de execução do TaskTracker em Python.

4.3 Plano de aprendizado para Git e GitHub

Na Fase 2, pretendo aprender a transformar este planejamento em um projeto versionado de forma organizada. Para isso, vou praticar a criação de um repositório no GitHub, entender o que é um commit e como escrever mensagens claras, salvar alterações de forma incremental, criar branches quando houver novas funcionalidades, resolver conflitos simples de código, acompanhar o histórico de versões e publicar o projeto de forma que outras pessoas consigam entender sua estrutura. A meta é usar Git e GitHub não apenas como local de armazenamento, mas como parte do processo de desenvolvimento e evolução do TaskTracker.

Tópicos que serão praticados: commit, push, pull, branch, histórico de versões, resolução de conflitos, README e publicação do repositório no GitHub.

5. Link da Apresentação em Vídeo

Após a gravação e publicação no YouTube como Público ou Não Listado, inserir abaixo o link ativo do vídeo de apresentação:

LINK DO YOUTUBE: https://youtu.be/EUj8gG6z5y4